

Transforming a Restaurant Application with AWS Analytics: From Static Hosting to Data-Driven Insights

Client Business Overview:

The client owns a restaurant who has a dedicated development team and instead of distributing physical paper menus to the customer who come to dine in the restaurant, he has put the QR codes on each table of his restaurant. QR code leads them to a static website hosted on Amazon S3. Customers choose the item they want to order from that menu. Then a waiter has to come to note down their order and then rest of cooking and food serving process is started.

Issues Facing:

- Customers must wait to order to be get noted by a waiter after they have chosen the items to order form the menu.
- Due to less staff restaurant's reputation is getting effected on their routine work.
- Delayed orders and delivery causes disturbance for the customer as well as staff.

Requirements:

Functional requirements:

1. The system must provide an **HTTPS RESTful endpoint** to ingest clickstream (analytics) data from the customers.
2. The endpoint should be exposed via a **managed AWS service** (e.g., Amazon API Gateway) rather than a self-managed server.
3. The solution must focus on the **data analytics pipeline** only (ingestion, storage, processing, analysis, reporting).

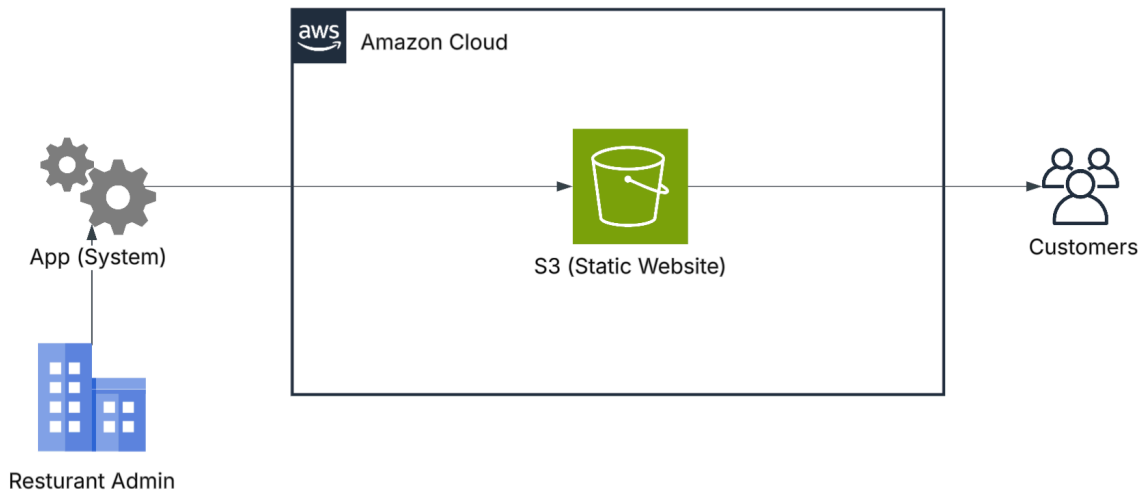
Non-functional / technical requirements:

1. Use **AWS managed, serverless services** wherever possible.
2. Use **cost-effective, usage-based billing** (pay per use), not time-based (e.g., avoid always-on servers).
3. Ensure **data durability**, including **backups to a different AWS Region**.
4. Ensure **encryption in transit** (HTTPS/TLS) and **encryption at rest** (e.g., KMS-encrypted storage).
5. The architecture should be **scalable** and **reliable** to handle varying traffic.

Current Architecture:

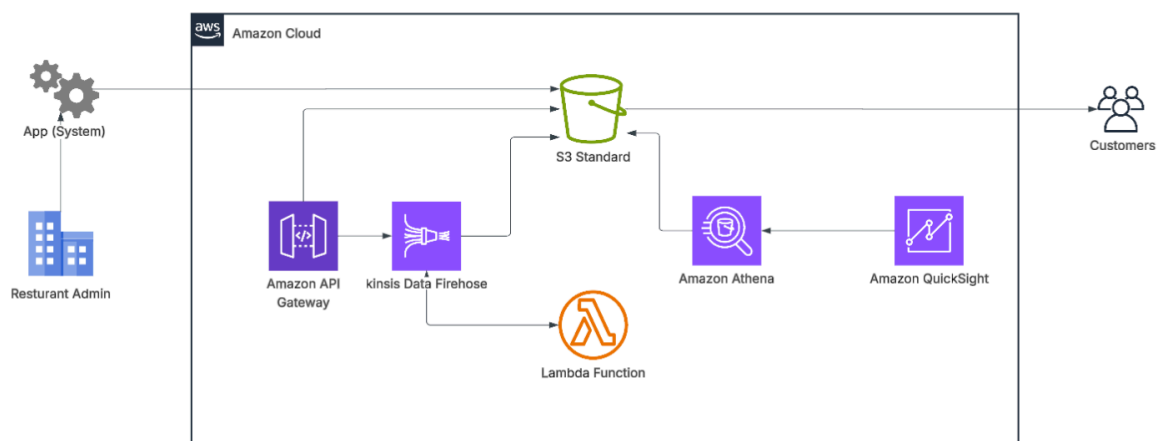
Following is the existing architecture of the application. This architecture uses Amazon S3 Static Website Hosting inside an AWS Virtual Private Cloud (VPC) to deliver the restaurant application.

The Restaurant Admin manages the application and uploads website content to the S3 bucket. Customers directly access the static website hosted on Amazon S3 through the internet.



Proposed Solution:

Following is an improved and more scalable analytics architecture. The architecture introduces Amazon API Gateway to securely collect application or customer request data. Amazon Kinesis Data Firehose streams the incoming data in near real time to Amazon S3 for centralized storage. AWS Lambda transforms data before Amazon Kinesis Data Firehose ingests it into the object storage bucket. Amazon Athena allows SQL-based querying directly on the data stored in S3 without managing servers. Amazon QuickSight creates dashboards and visual reports for business insights and customer trends. Customers continue using the application normally while analytics run in the background. Restaurant Admins can monitor sales, customer activity, and system performance through dashboards. Overall, this architecture improves real-time analytics, scalability, automation, and business intelligence capabilities.



AWS Services Used:

AWS Service	Purpose in the Architecture
Amazon API Gateway	Acts as the entry point for the application and securely receives API requests, user activity, and transaction data from the system.
Amazon Kinesis Data Firehose	Collects and streams real-time data from the application to storage services for analytics processing.
AWS Lambda	Transforms data before Amazon Kinesis Data Firehose ingests it into the object storage bucket
Amazon S3 (Simple Storage Service)	Stores application data, logs, analytics data, and processed files in a scalable and durable storage system.
Amazon Athena	Allows SQL queries directly on data stored in S3 without needing a separate database server.
Amazon QuickSight	Creates custom dashboards, charts, and visual analytics reports for monitoring business insights and customer trends.
Amazon Cloud / AWS Environment	Provides the secure and scalable cloud infrastructure where all services are hosted and connected.

Security Considerations:

- API Security (Amazon API Gateway)
- Data Validation and Sanitization
- Network Protection
- Data Encryption
- Access Control and IAM Policies
- Secure S3 Buckets
- Lambda Function Security
- Network Protection
- Backup and Disaster Recovery
- Compliance and Privacy

Business Outcome Matrix:

Business Outcome	Related Metrics / KPIs
Improved Customer Experience	Customer satisfaction score, response time, website availability, user engagement
Better Decision Making	Accuracy of reports, dashboard usage, real-time analytics availability
Increased Operational Efficiency	Reduced manual reporting time, automation rate, faster data processing
Scalability and Performance	Number of users supported, system uptime, data processing speed
Revenue Growth	Increase in sales, repeat customers, average order value
Cost Optimization	Reduced infrastructure cost, serverless resource utilization, storage efficiency
Real-Time Insights	Time taken to generate reports, live monitoring capability
Data Reliability and Availability	Data durability, backup success rate, analytics data availability
Security and Compliance	Number of security incidents, audit compliance status, access control effectiveness

Business Intelligence Enhancement	Quality of visual dashboards, trend analysis accuracy, forecasting capability
--	---

Cost Optimization Benefits:

AWS Service	Cost Optimization Strategy
Amazon API Gateway	Use throttling and caching to reduce unnecessary API requests and lower request-processing costs.
Kinesis Data Firehose	Configure buffering intervals and compression to reduce data transfer and storage costs.
AWS Lambda	Optimize function memory and execution time to avoid paying for unused compute resources.
Amazon S3	Use lifecycle policies to move old data to cheaper storage classes like S3 Glacier or Intelligent-Tiering.
Amazon Athena	Store data in compressed columnar formats (Parquet/ORC) to reduce query scan costs.
Amazon QuickSight	Use SPICE caching efficiently and limit unnecessary dashboard refresh frequency.
Monitoring Services	Retain logs only for required durations to reduce CloudWatch storage charges.
Data Transfer Costs	Minimize cross-region traffic and keep services within the same AWS region when possible.
Serverless Architecture	Use pay-as-you-go services like Lambda and Athena to avoid maintaining always-running servers.
Resource Tagging and Budgets	Use AWS Budgets and cost allocation tags to monitor spending and identify unused resources.

Final Thoughts:

This enhanced AWS analytics architecture is a strong improvement over the earlier basic hosting setup because it introduces real-time data processing, scalable storage, and business intelligence capabilities.

By integrating services such as API Gateway, Kinesis Firehose, Lambda, Athena, and QuickSight, the system becomes more automated, data-driven, and scalable.

The architecture enables restaurant administrators to gain valuable insights into customer behavior, sales trends, and operational performance through interactive dashboards and analytics.

Using serverless and managed AWS services also reduces infrastructure management overhead while improving flexibility and cost efficiency.

With proper security controls, monitoring, and cost optimization strategies, this solution can support future business growth while maintaining reliability, performance, and secure data handling.

Architecting Solutions: Building a Proof of Concept for Data Analytics

Task 1: Setup: Creating the IAM policy and role

Task 1.1: Creating custom IAM policies

The screenshot shows the AWS IAM console interface. On the left is a navigation menu with sections like 'Identity and Access Management (IAM)', 'Access Management', and 'Access reports'. The main content area displays the 'API-Firehose' policy details. A green banner at the top indicates 'Policy API-Firehose created.' with a 'View policy' button. Below this, the 'Policy details' section shows the policy type as 'Customer managed', creation and edited times as 'May 29, 2026, 09:37 (UTC+01:00)', and the ARN as 'arn:aws:iam::[redacted]:policy/API-Firehose'. The 'Permissions' tab is selected, showing the JSON policy document. The document defines a single permission with 'Sid': 'VisualEditor0', 'Effect': 'Allow', 'Action': 'firehose:PutRecord', and 'Resource': '*'. The bottom of the console shows a footer with 'CloudShell', 'Feedback', 'Console Mobile App', and copyright information for Amazon Web Services, Inc.

Identity and Access Management (IAM)

Search IAM

Dashboard

▼ Access Management

- Roles
- Policies**
- IAM users
- IAM user groups
- Identity providers
- Account settings
- Root access management
- Temporary delegation requests

▼ Access reports

- Access Analyzer
 - Resource analysis
 - Unused access
 - Analyzer settings
- Credential report
- Organization activity
- Service control policies

Policy API-Firehose created. View policy

API-Firehose Info Edit Delete

Policy details

Type	Creation time	Edited time	ARN
Customer managed	May 29, 2026, 09:37 (UTC+01:00)	May 29, 2026, 09:37 (UTC+01:00)	arn:aws:iam::[redacted]:policy/API-Firehose

Permissions Entites attached Tags Policy versions (1) Last Accessed

Permissions defined in this policy Info Copy Edit Summary JSON

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Sid": "VisualEditor0",
6       "Effect": "Allow",
7       "Action": "firehose:PutRecord",
8       "Resource": "*"
9     }
10  ]
11 }
```

CloudShell Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Task 1.2: Creating an IAM role and attaching a policy to it

The screenshot shows the AWS IAM console interface for the 'APIGateway-Firehose' role. A green banner at the top indicates 'Role APIGateway-Firehose created.' with a 'View role' button. The main content area displays the role details. The 'Summary' section shows the creation date as 'May 29, 2026, 09:41 (UTC+01:00)', the ARN as 'arn:aws:iam::[redacted]:role/APIGateway-Firehose', and the maximum session duration as '1 hour'. The 'Permissions' tab is selected, showing a list of attached policies. One policy, 'AmazonAPIGatewayPushToCloudWatchLogs', is listed with the type 'AWS managed'. The bottom of the console shows a footer with 'CloudShell', 'Feedback', 'Console Mobile App', and copyright information for Amazon Web Services, Inc.

Identity and Access Management (IAM)

Search IAM

Dashboard

▼ Access Management

- Roles**
- Policies
- IAM users
- IAM user groups
- Identity providers
- Account settings
- Root access management
- Temporary delegation requests

▼ Access reports

- Access Analyzer
 - Resource analysis
 - Unused access
 - Analyzer settings
- Credential report
- Organization activity
- Service control policies

Role APIGateway-Firehose created. View role

APIGateway-Firehose Info Delete

Allows API Gateway to push logs to CloudWatch Logs.

Summary Edit

Creation date	ARN
May 29, 2026, 09:41 (UTC+01:00)	arn:aws:iam::[redacted]:role/APIGateway-Firehose
Last activity	Maximum session duration
-	1 hour

Permissions Trust relationships Tags Last Accessed Revoke sessions

Permissions policies (1) Info Simulate Remove Add permissions

You can attach up to 10 managed policies.

Search Filter by Type All types

Policy name	Type	Attached entities
AmazonAPIGatewayPushToCloudWatchLogs	AWS managed	1

CloudShell Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

[Cancel](#) [Add permissions](#)

<

Dashboard

- Roles
- Policies
- IAM users
- IAM user groups
- Identity providers
- Account settings
- Root access management
- Temporary delegation requests

▼ Access reports

- Access Analyzer
 - Resource analysis
 - Unused access
 - Analyzer settings
- Credential report
- Organization activity
- Service control policies

[Simulate ↗](#)
[Remove](#)
[Add permissions ▼](#)

All types

Policy name ↗

▲ | Type

Attached ... ▾

01

 AmazonAPIGatewayPushToCloudWatchLogs

AWS managed

1

1

[+ API-Firehose](#)

Customer managed

1

You can generate a new policy based on the access activity for this role, then customize, create, and attach it to this role. AWS uses your CloudTrail events to identify the services and actions used and generate a policy. [Learn more](#)

Consent to publish:

No requests to gene

No requests to generate a policy in the past 7 days.

Task 2: Creating an S3 bucket

Amazon S3 > Buckets > architecting-week2-adi

Successfully created bucket "architecting-week2-adi"

architecting-week2-adi info

Objects | Metadata | Properties | Permissions | Metrics | Management | File systems - new | Access Points

Objects (0)   Copy S3 URI  Copy URL  Download  Open  Delete  Actions  Create folder  Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

Name	Type	Last modified	Size	Storage class
No objects				
You don't have any objects in this bucket.				

 Upload

Task 3: Creating a Lambda function

Lambda > Functions > transform-data

Successfully created the function "transform-data". You can now change its code and configuration. To invoke your function with a test event, choose "Test".

Code source info

 Open in Visual Studio Code  Update 

transform-data

EXPLORER

- TRANSFORM-DATA
 - __pycache__
 - lambda_function.py

DEPLOY Undeployed

Deploy (🔗U)

Test (🔗%)

TEST EVENTS [NONE SELECTED]

+ Create new test event

```
1 import json
2 import boto3
3 import base64
4
5 output = []
6
7 def lambda_handler(event, context):
8
9     for record in event['records']:
10         payload = base64.b64decode(record['data']).decode('utf-8')
11
12         row_w_newline = payload + "\n"
13         row_w_newline = base64.b64encode(row_w_newline.encode('utf-8'))
14
15         output_record = {
16             'recordId': record['recordId'],
17             'result': 'OK',
18             'data': row_w_newline
19         }
20         output.append(output_record)
21
22     return {'records': output}
```

Amazon Q Tip 1/3: Start typing to get suggestions ([ESC] to exit)

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Task 4: Creating a Kinesis Data Firehose delivery stream

Task 4.1: Creating the Kinesis Data Firehose delivery stream